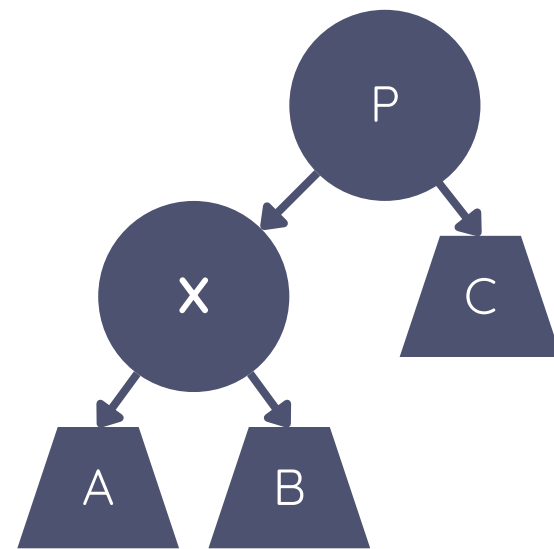
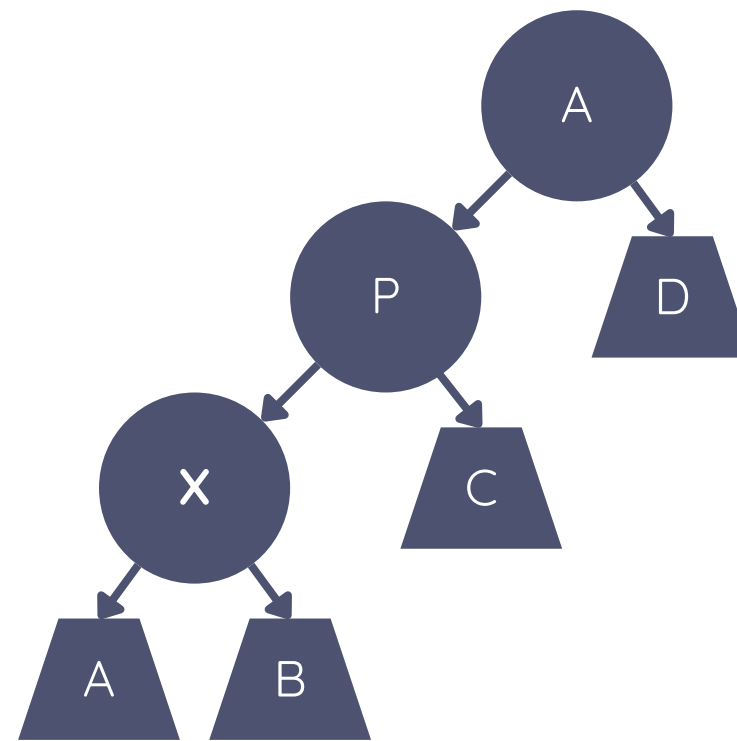


PRE CONTROL 2

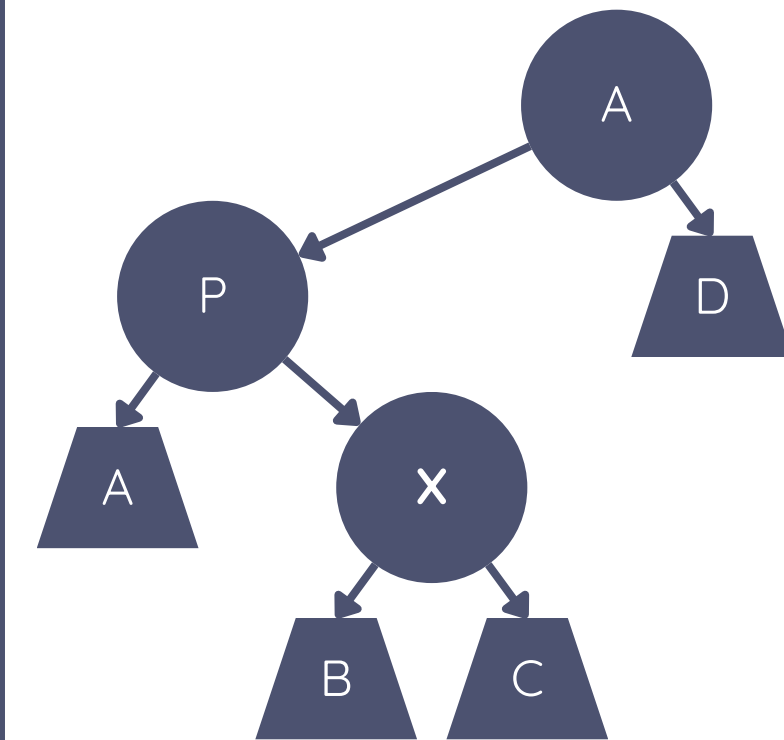
Zig



Zig-Zig

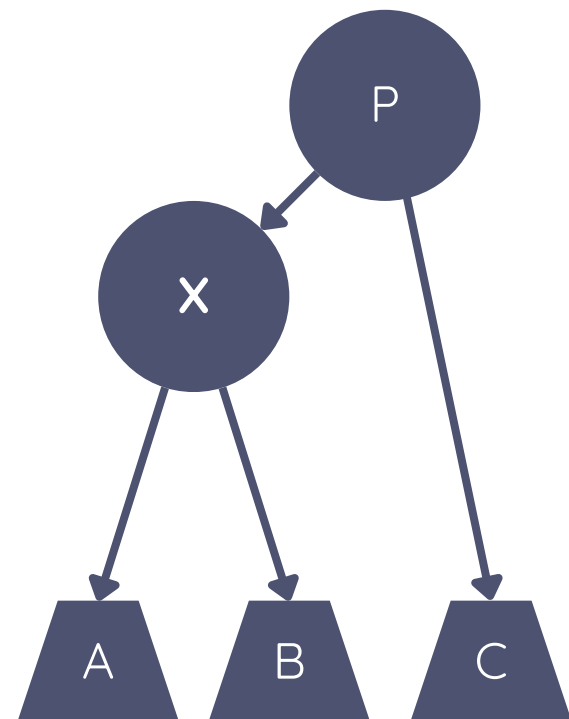


Zig-Zag

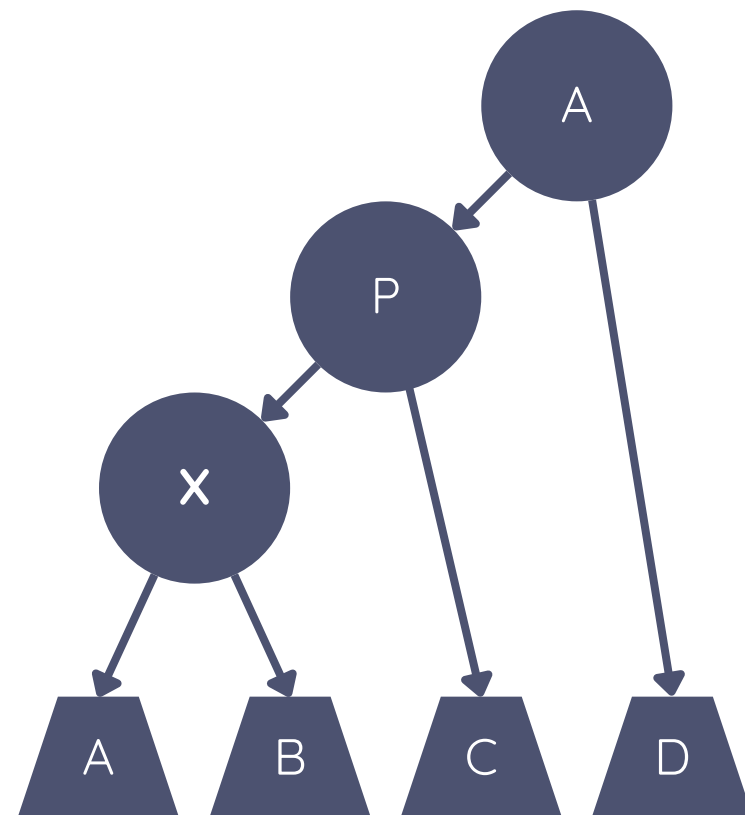


ROTACIONES EN ARBOLES

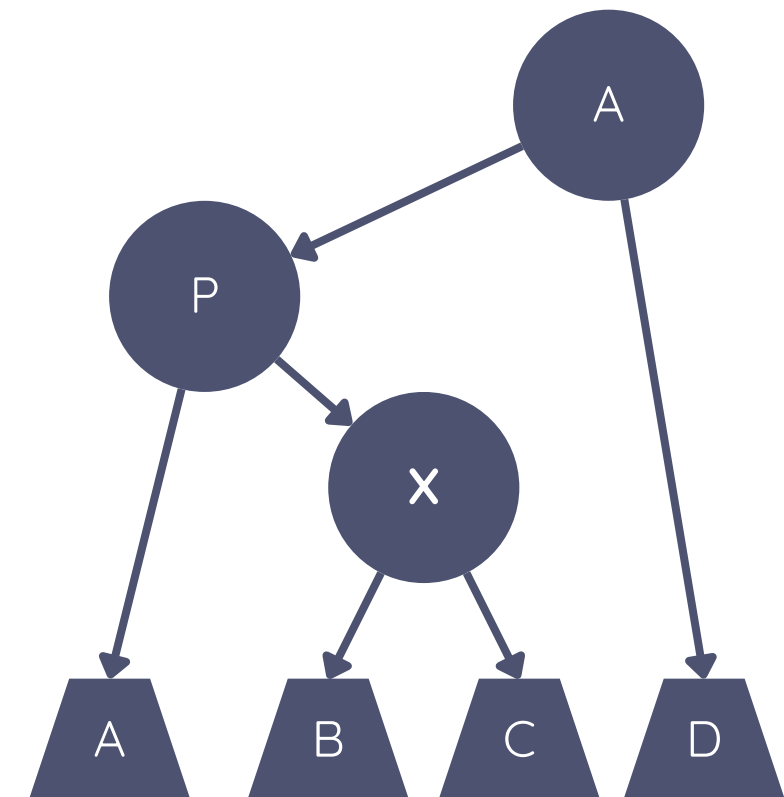
Zig



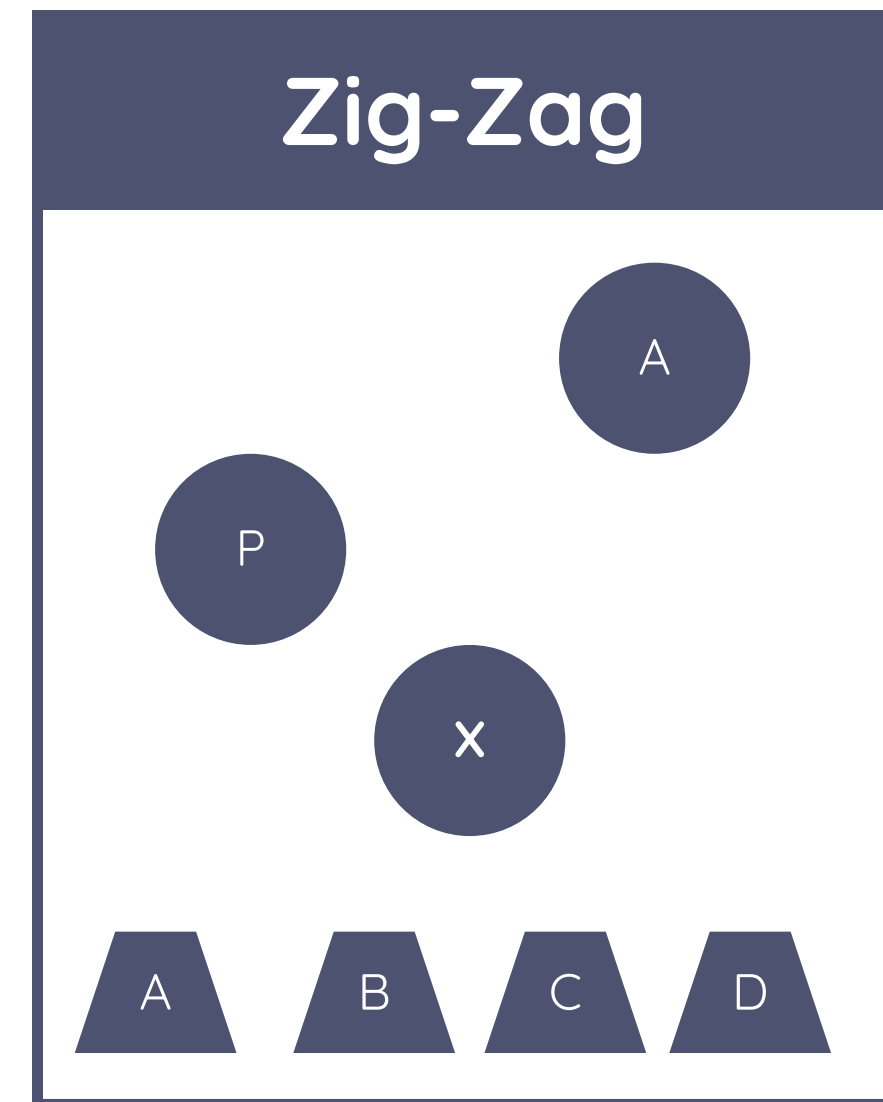
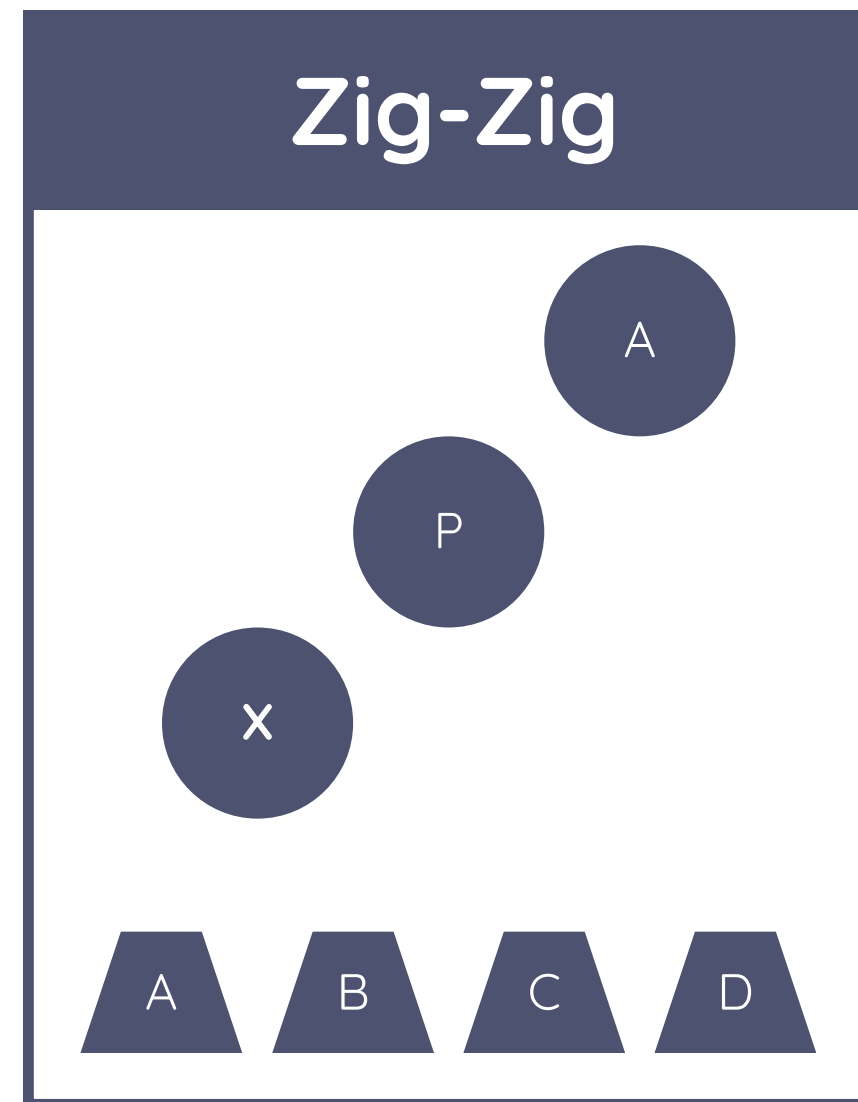
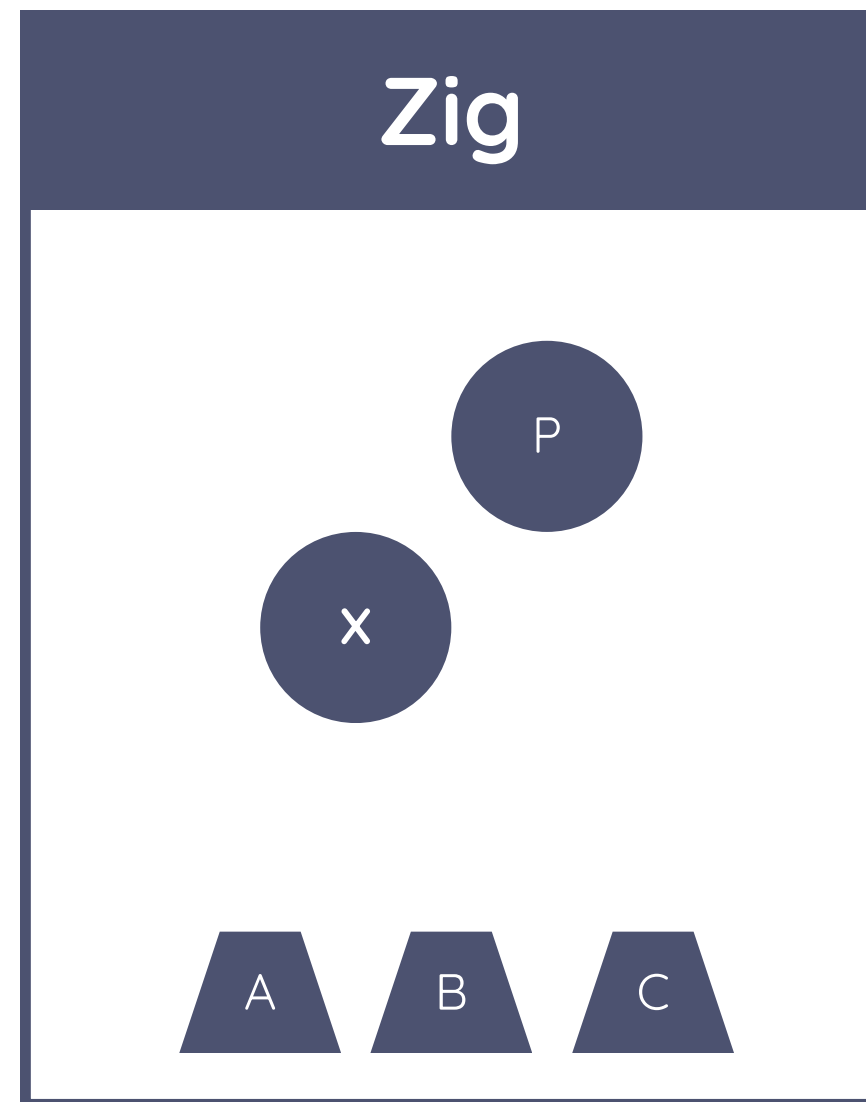
Zig-Zig



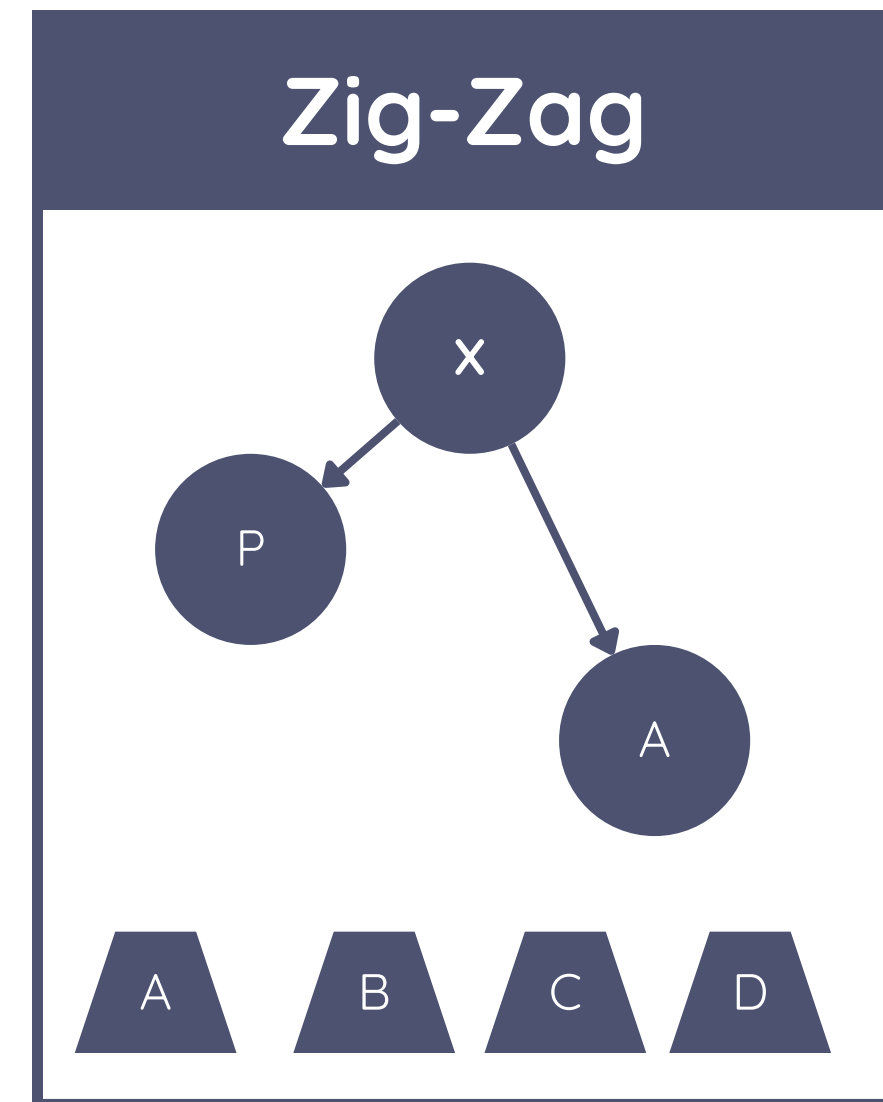
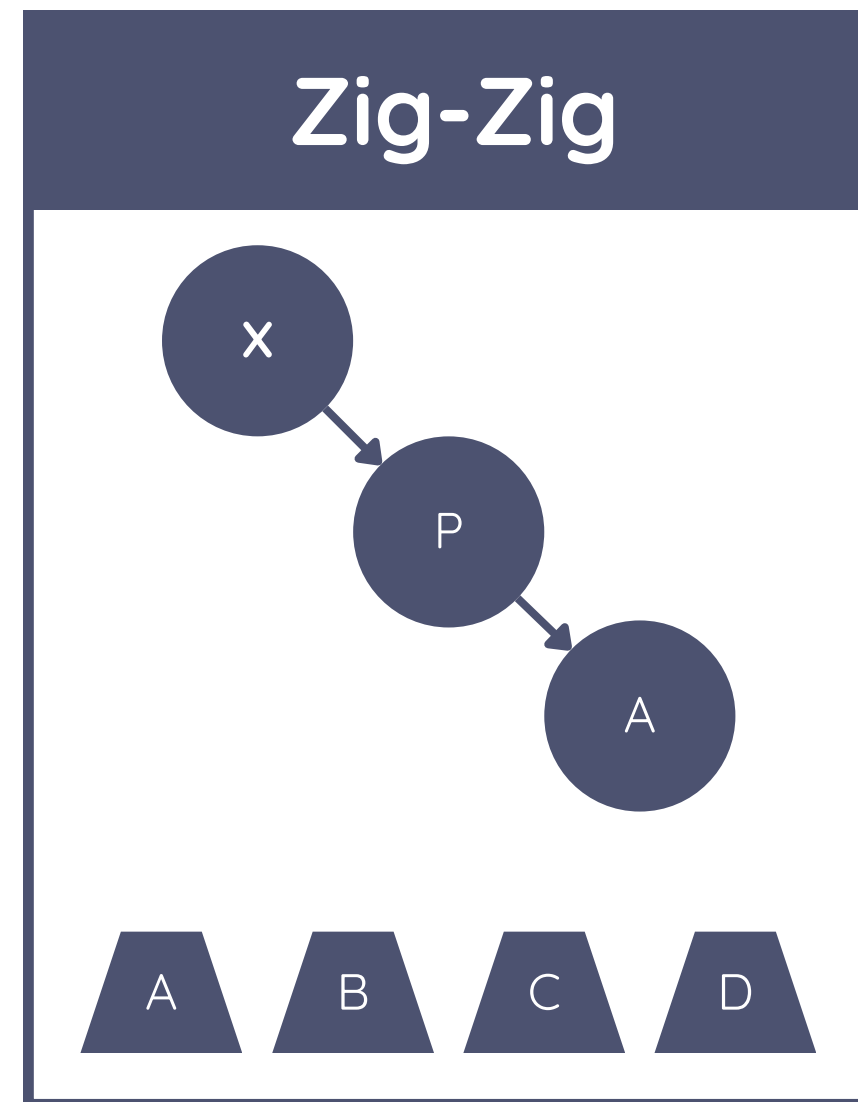
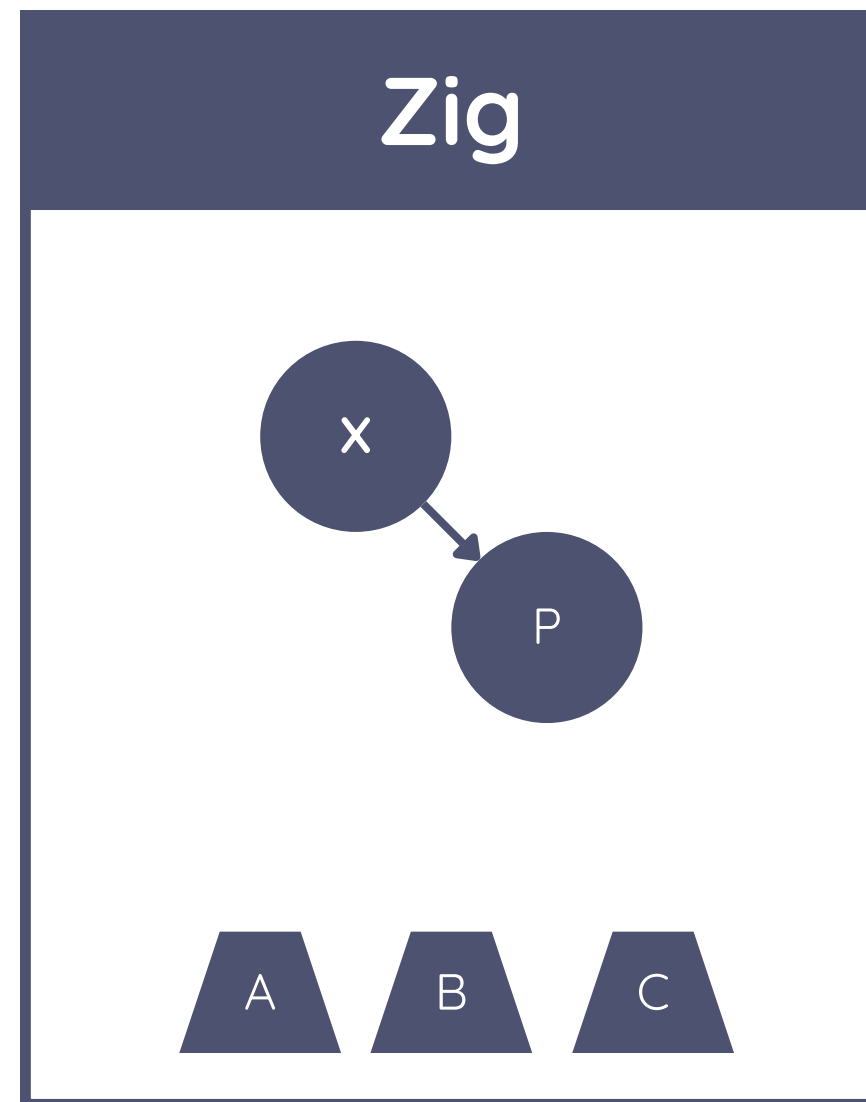
Zig-Zag



ROTACIONES EN ARBOLES

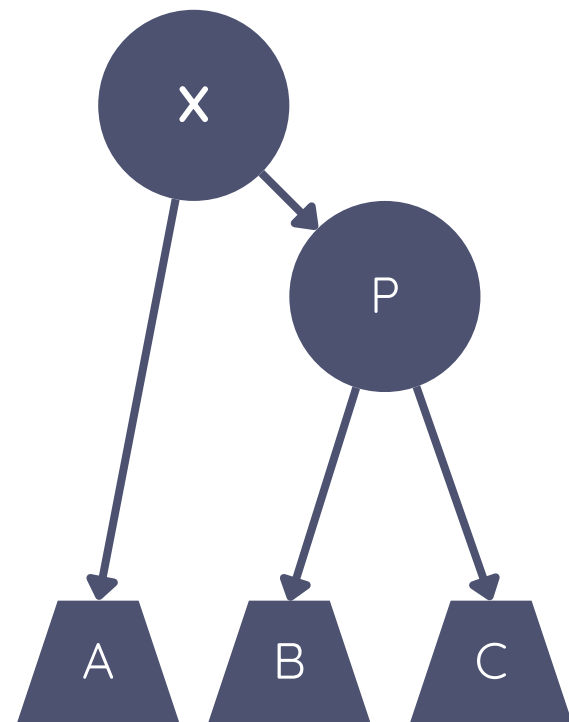


ROTACIONES EN ARBOLES

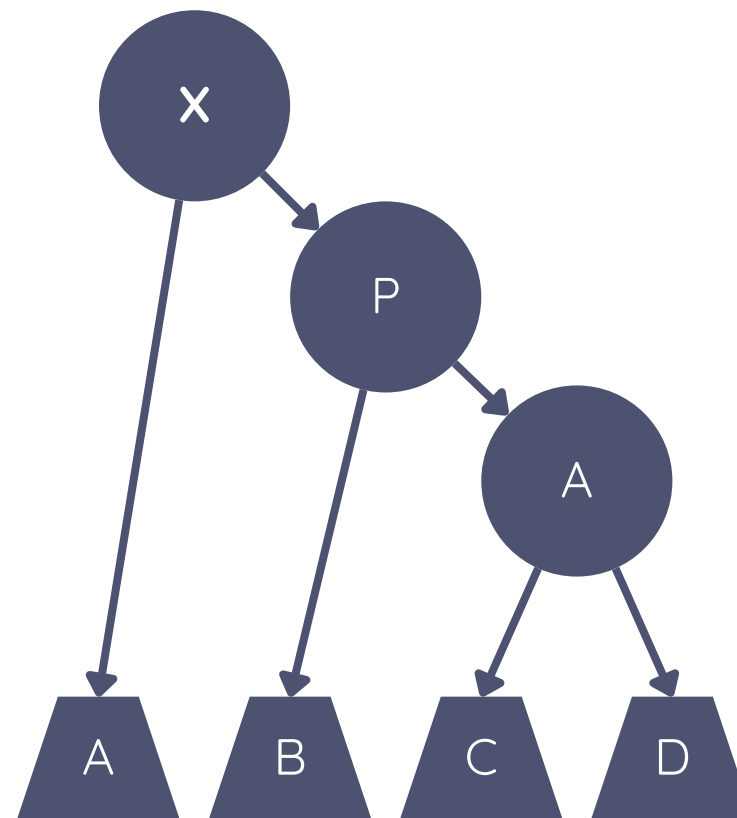


ROTACIONES EN ARBOLES

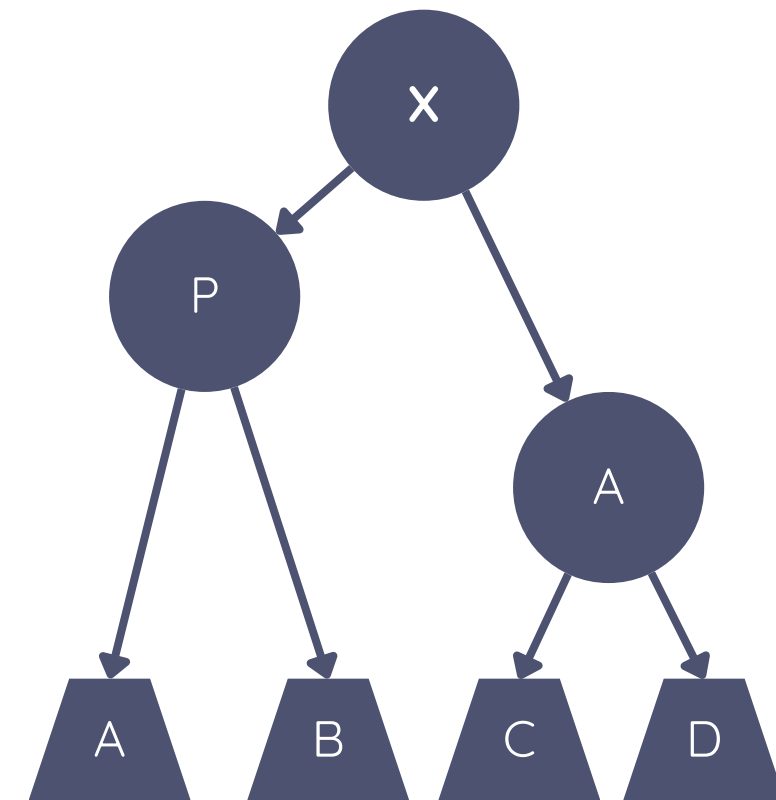
Zig



Zig-Zig

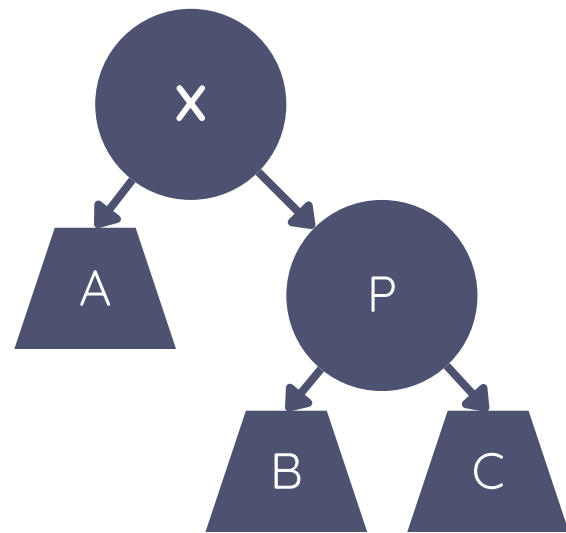


Zig-Zag

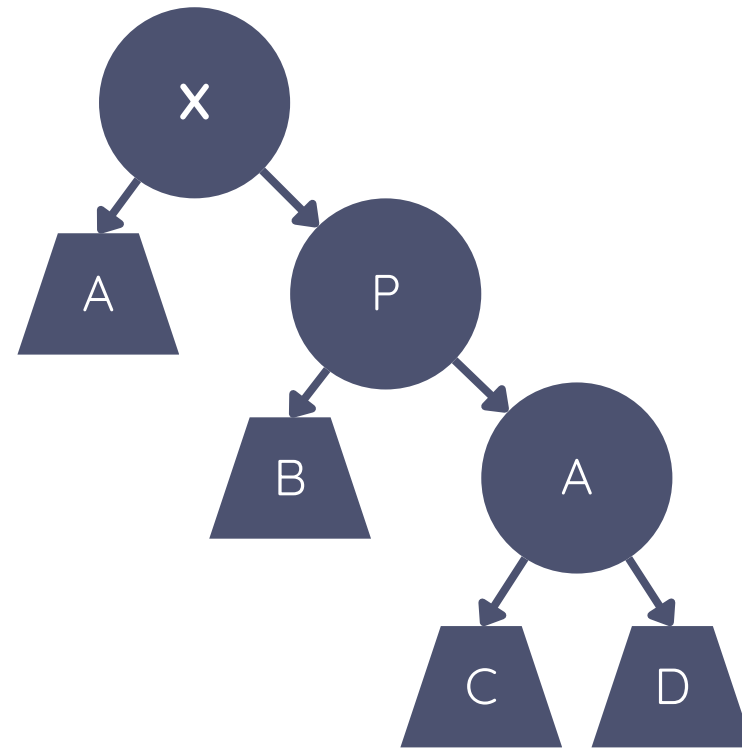


ROTACIONES EN ARBOLES

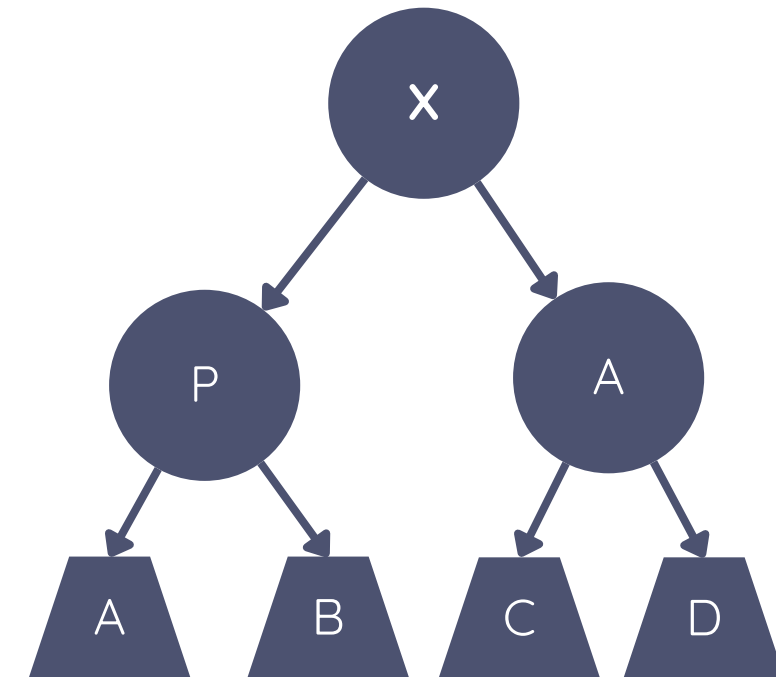
Zig



Zig-Zig

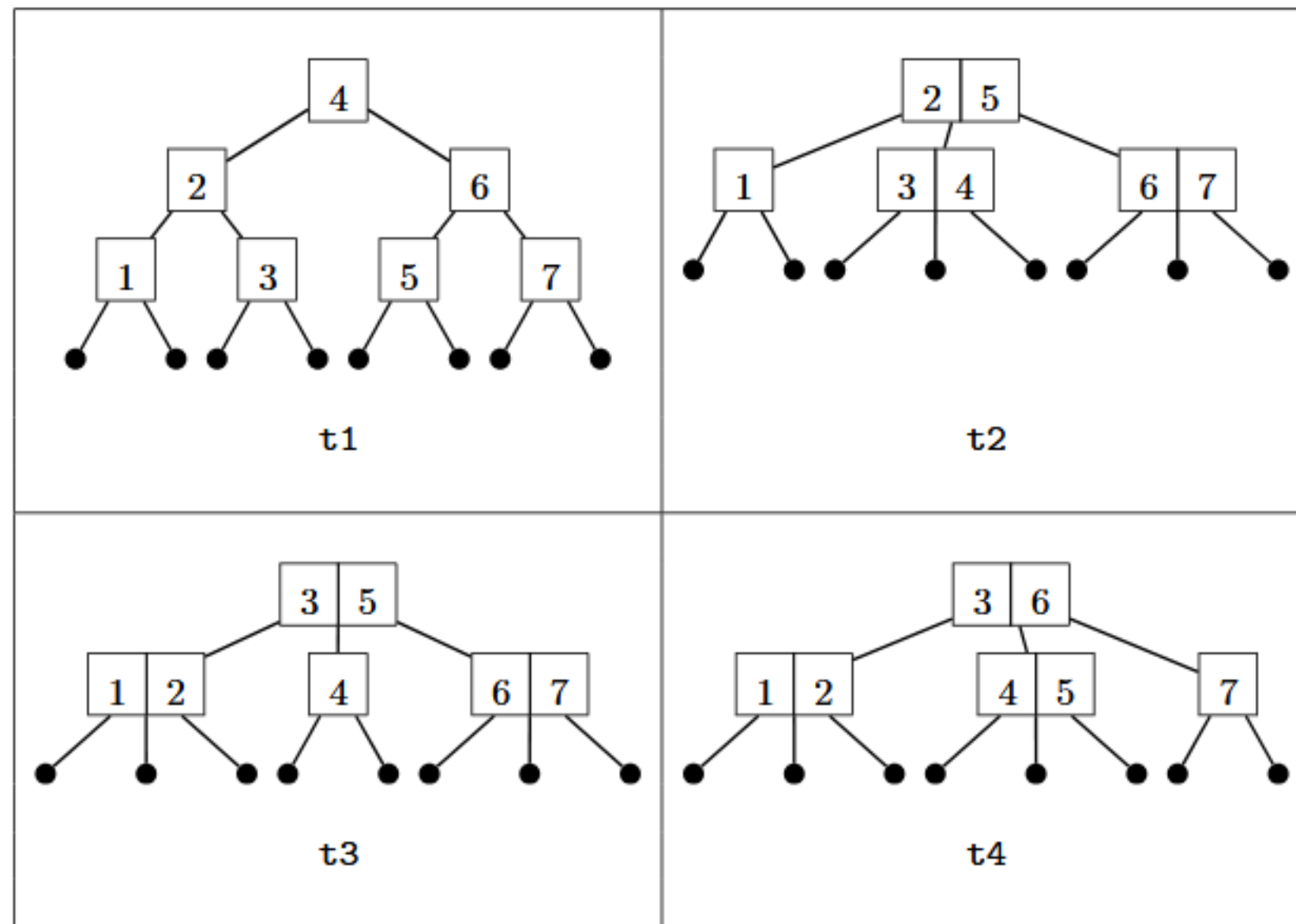


Zig-Zag



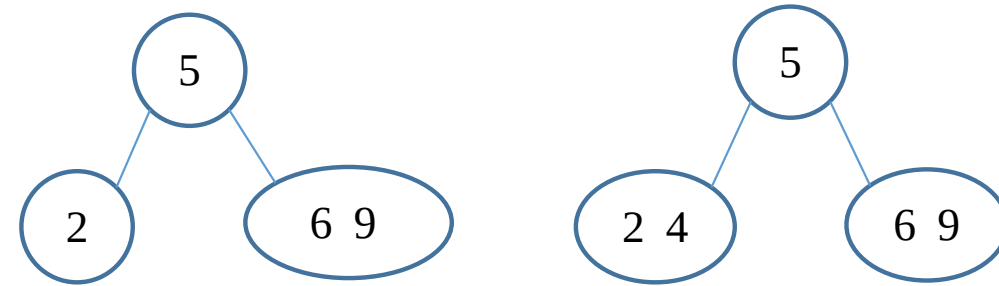
ROTACIONES EN ARBOLES

Un árbol 2-3 es un árbol de búsqueda el cual se caracteriza por tener nodos con 2 o 3 hijos, y dependiendo puede tener 1 o 2 llaves por nodo.

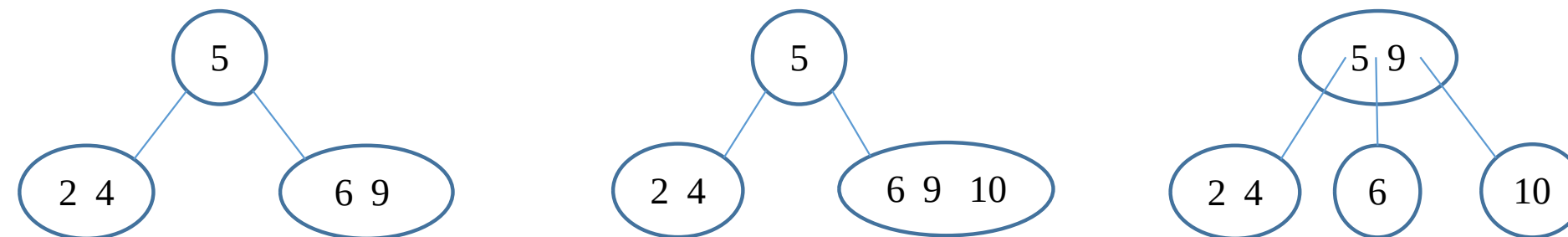


ARBOLES 2-3

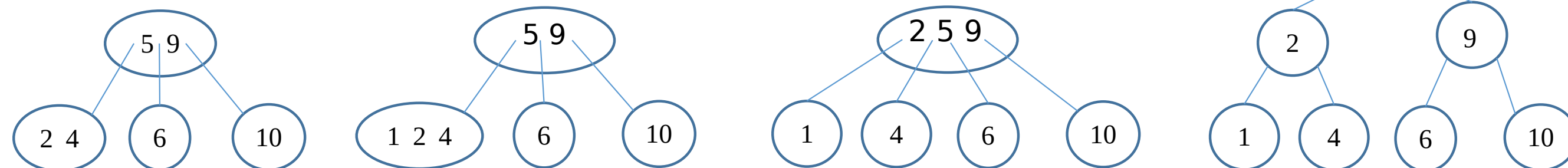
Insert in a 2-node :



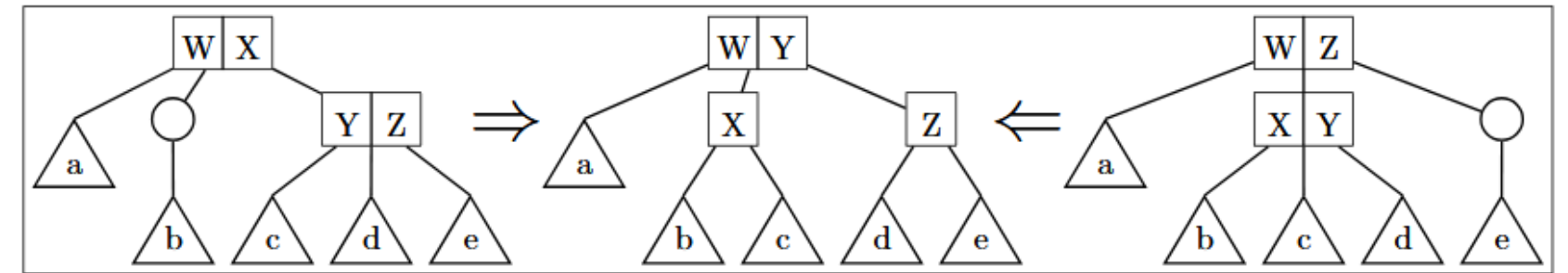
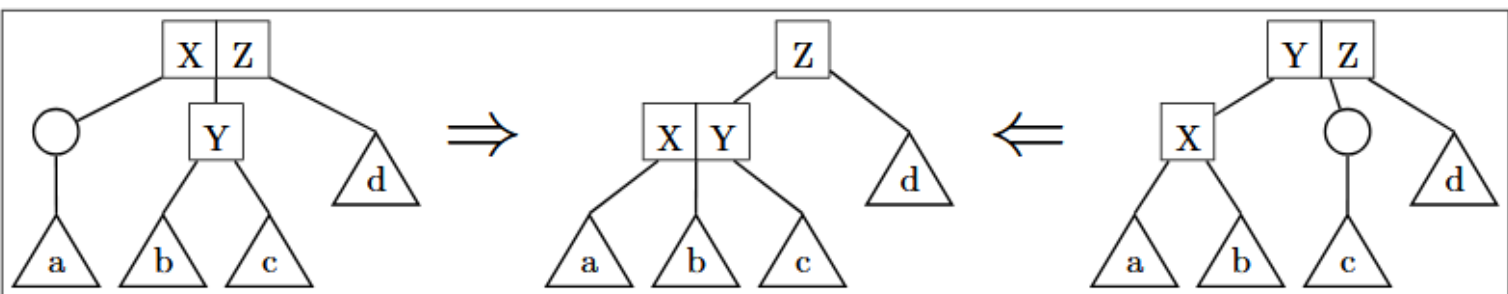
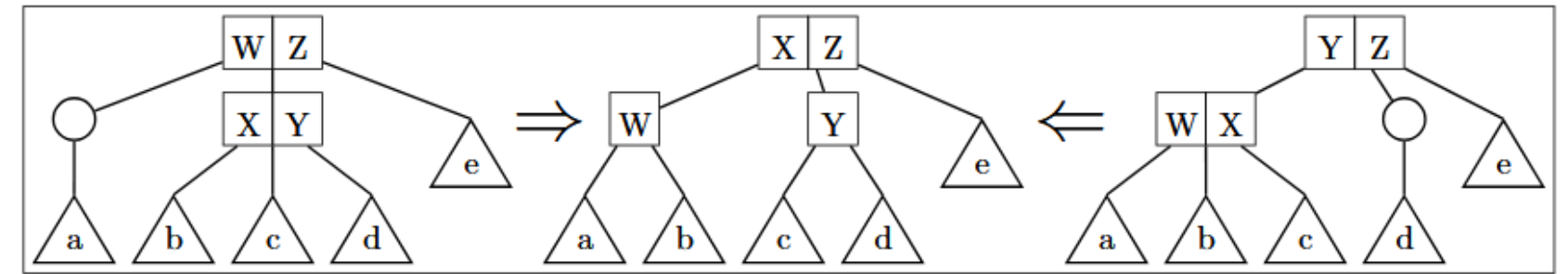
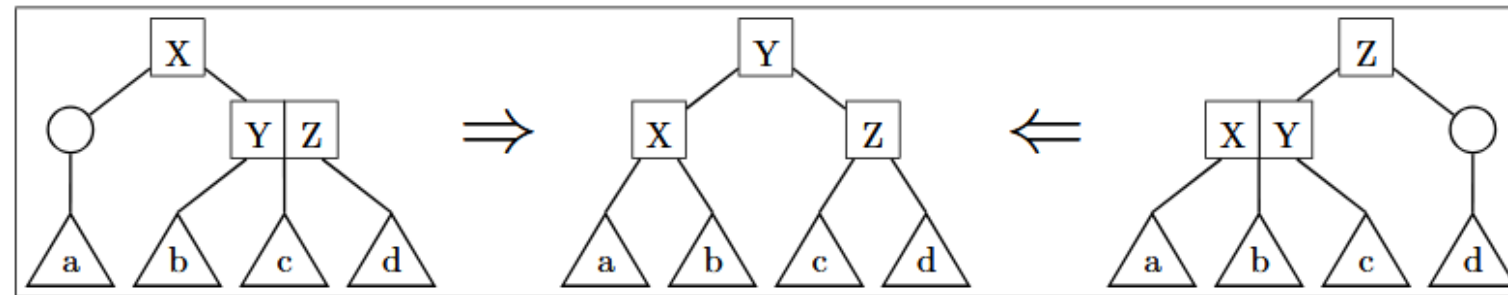
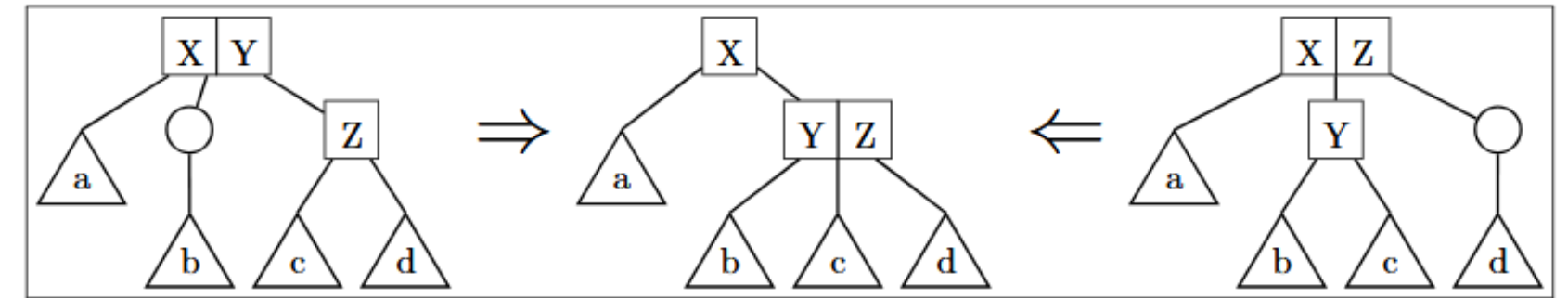
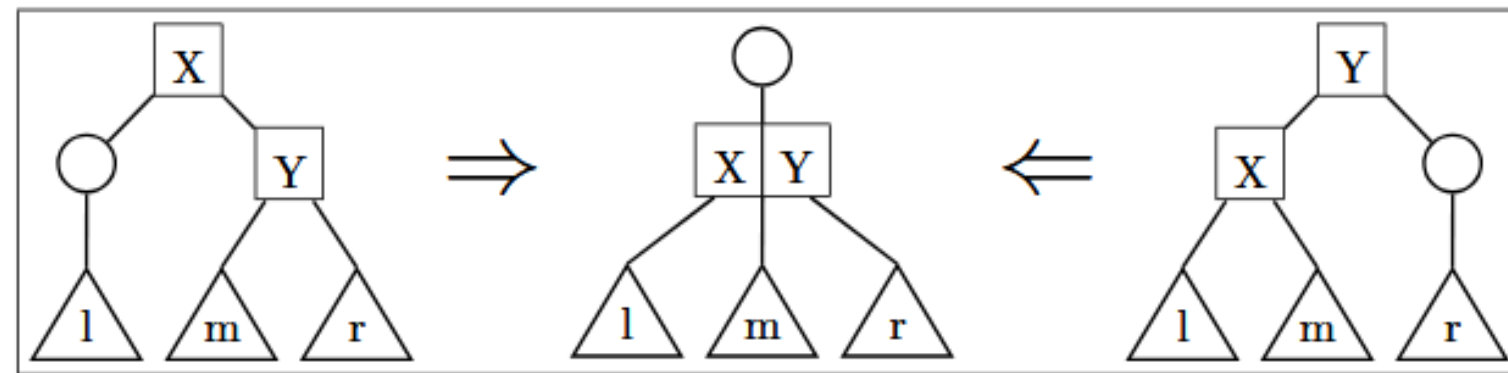
Insert in a 3-node (2 node parent) :



Insert in a 3-node (3 node parent) :



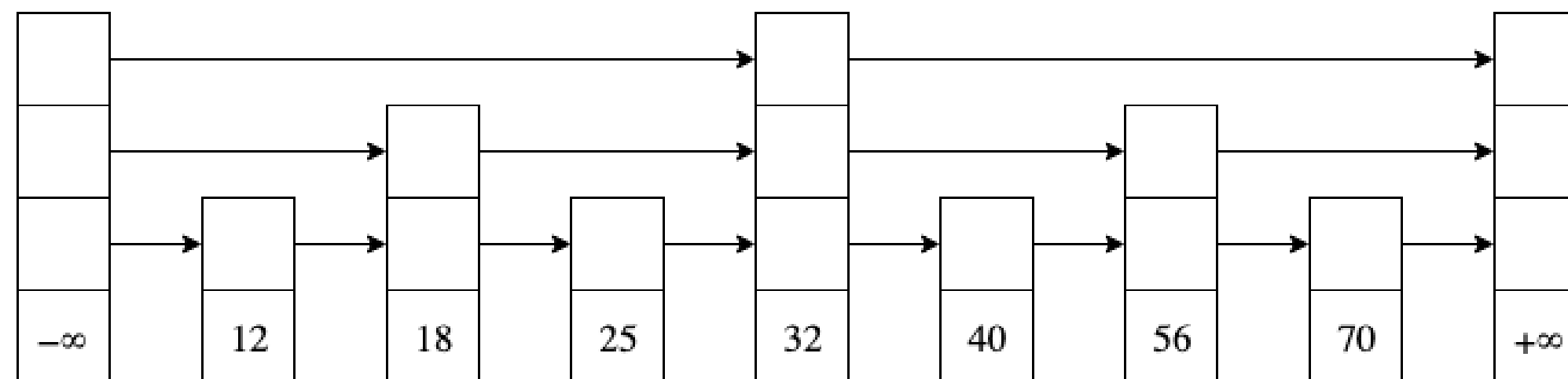
INSERCIÓN EN 2-3



<https://www.cs.princeton.edu/~dpw/courses/cos326-12/ass/2-3-trees.pdf>

ELIMINACION EN 2-3

Un Skip list es una lista enlazada ordenada la cual nosotros agregamos mas punteros hacia nodos lejanos, para poder saltarnos a una cantidad significativa, hay muchas formas de asignar estas torres de punteros, pero la idea principal se mantiene, de poder tener alguna forma de saltar informacion de forma rapida para poder obtener busquedas eficientes

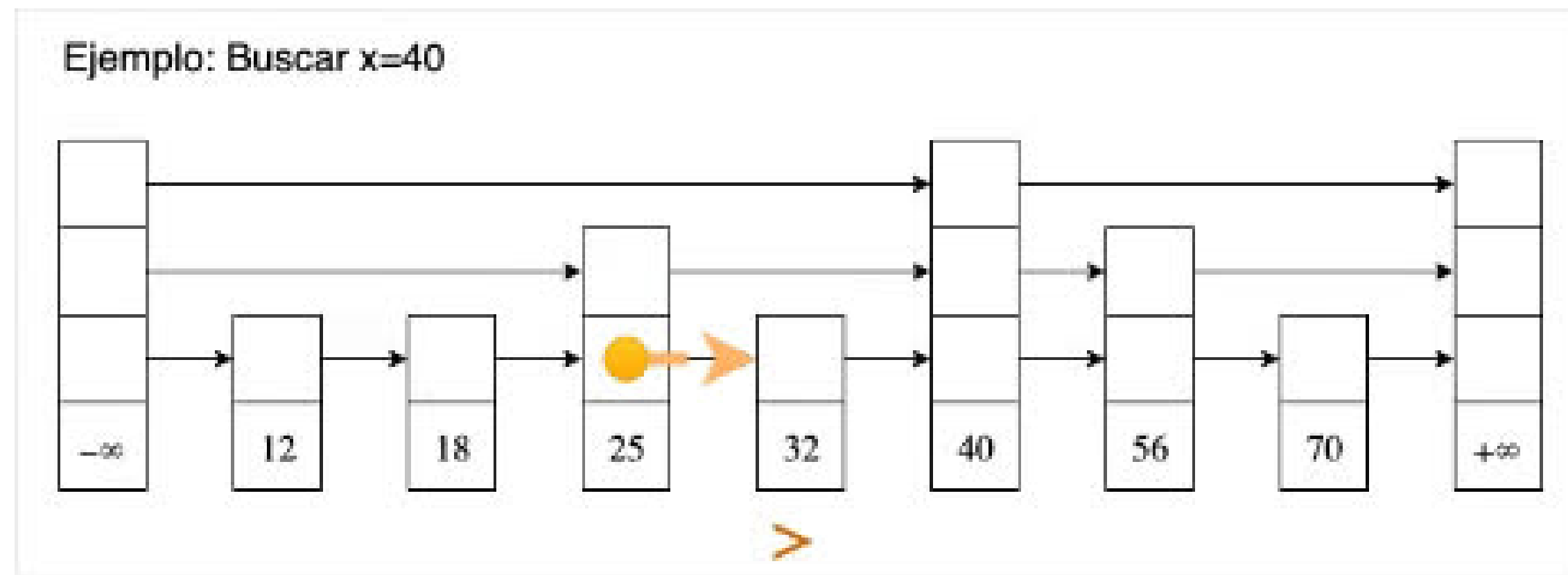


SKIP LISTS

Se basa en dos principios. Dado que se busca X , partimos desde el puntero mas lejano Y

- Si $X > Y$, Se avanza
- Si $X \leq Y$, Bajamos de puntero Skip y volvemos al primer criterio

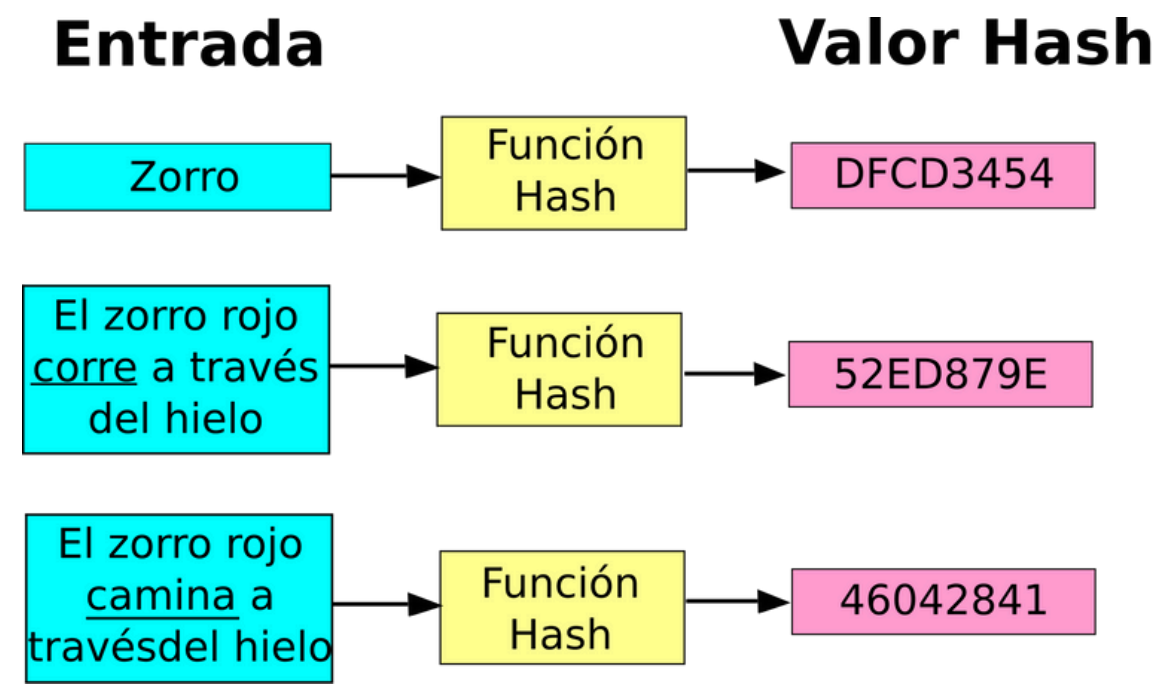
Cuando ya no podemos aplicar el ultimo criterio vemos si el siguiente nodo $X = Y$ sino no esta



BUSQUEDA EN SKIP LIST

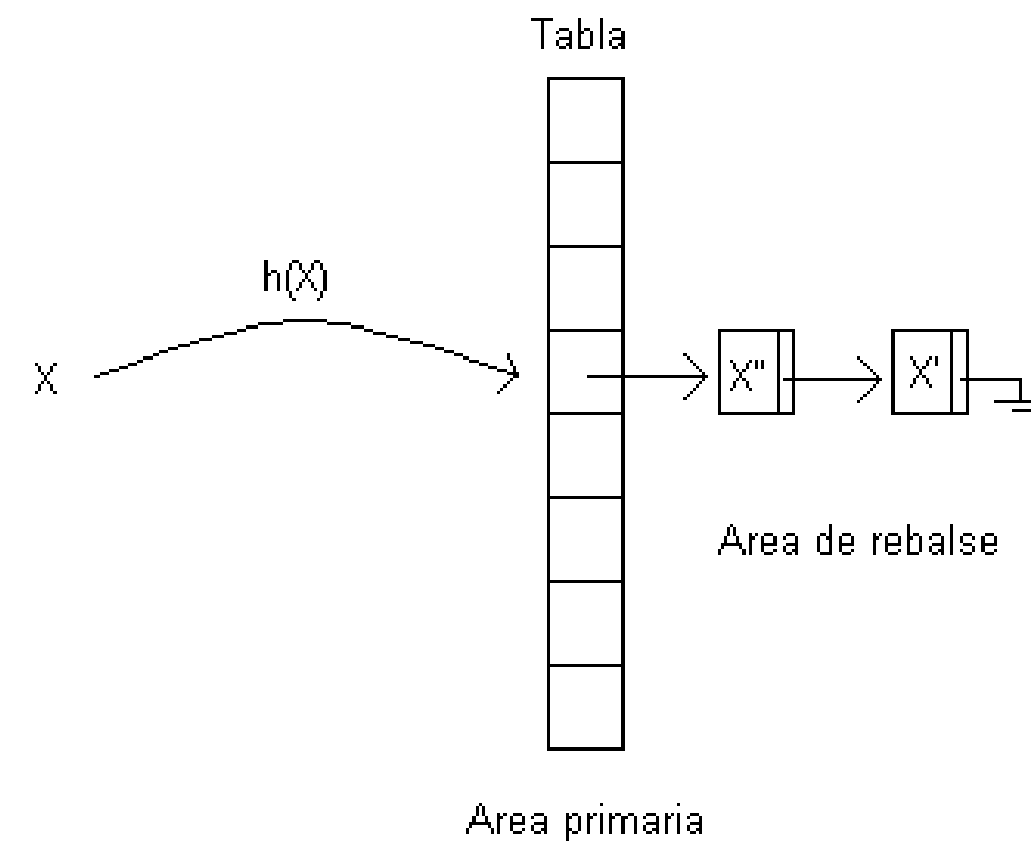
Hashing se basa en una función $h(x)$ que mapea de 0 a $m-1$, que podemos tabular, con la cual buscamos una representación en teoría, única de cualquier elemento x en nuestro dominio.

Problema (palomar), esto no puede realmente llegar a suceder en un codominio finito menor al dominio, y ocurren colisiones hay distintas formas de resolver este problema, uno de ellos es el encadenamiento



HASHING: ENCADE- NAMIENTO

Dado algun elemento X , tomamos su $h(X)$, e intentamos introducirlo en el indice de la tabla, si la paradoja del cumpleaños ataco y encontramos otro elemnto Y tal que $h(Y) = h(X)$ anterior, entonces tomaremos la opcion de encadenar, y diremos que se abre una lista de revalse, en la cual el primer nodo de la lista es el inicial Y , y el ultimo el ultimo en insertarse X



ENCADENAMIENTO EN HASHING